Proyecto Grupal de Ingeniería de Software

Pipeline DataOps para la calidad de datos agroclimáticos IoT

Documento de Proyecto – Entrega final

Materia: Ingeniería de Software – UADE
Grupo / Integrantes: [Completar: N° de grupo y nombres]
Docente: [Completar]
Fecha de entrega: 10 de noviembre de 2026
Versión: [Completar]

1. Sobre la consultora (nosotros)

Nos posicionamos como una consultora especializada en Data Engineering y DataOps, contratada por la organización cliente para diseñar e implementar un proceso que automatice la ingesta, validación, transformación y monitoreo de los datos generados por sensores IoT, incorporando prácticas de CI/CD.

UnaCocuCha SRL es una consultora argentina de Data Engineering y DataOps con 5 años de experiencia acompañando a empresas medianas y grandes a convertir datos operativos en información confiable. Trabajamos con equipos chicos y multidisciplinarios, y entregamos resultados en ciclos cortos.

Servicios:

- Diseño e implementación de pipelines de datos (ETL/ELT) con arquitectura Medallion.
- Data Quality: definición de reglas, validación automática y métricas de calidad.
- DataOps y CI/CD: versionado, tests automatizados y despliegue continuo de pipelines.
- Tableros de indicadores (Power BI / Grafana) sobre datos validados.

Enfoque de trabajo: empezamos con una prueba de concepto de pocas semanas, trabajamos con metodología ágil (tablero Kanban y reuniones semanales con el cliente), usamos herramientas open source para bajar costos de licencias y hacemos transferencia de conocimiento al equipo del cliente al cierre.

1.1 Equipo y roles

Integrante	Rol	Responsabilidades
[Nombre]	Project Manager	Planificación (EDT, Gantt), seguimiento del tablero Kanban, gestión de riesgos y costos, comunicación con el cliente y actas de reunión.
[Nombre]	Data Engineer	Ingesta desde Open-Meteo y simulador de sensores IoT; construcción de las capas Bronze, Silver y Gold; modelo de datos.
[Nombre]	QA / Data Quality	Definición de reglas y rangos junto al cliente, tests automatizados (Pytest), métricas de calidad (Data Quality Score) y dashboard de indicadores.
[Nombre]	DevOps / CI-CD	Repositorio Git y convenciones, pipeline de GitHub Actions, contenedores Docker, despliegue y manejo seguro de credenciales.

Nota: el equipo está formado por 4 integrantes; una misma persona puede colaborar en tareas de otro rol cuando el cronograma lo requiera.

2. Sobre el cliente

2.1 Descripción de la organización

El cliente es una empresa agropecuaria de gran escala que posee múltiples campos equipados con sensores IoT. Estos dispositivos generan grandes volúmenes de datos sobre temperatura, humedad del suelo, precipitaciones, humedad ambiental y condiciones meteorológicas.

Tipo de organización: Agropecuaria / Agroindustria.

Cliente (organización ficticia): Agropecuaria Los Tres Ombúes S.A.

- **Actividad:** agricultura extensiva (soja, maíz y trigo) y ganadería complementaria.
- **Superficie:** 6 campos que suman unas 18.000 hectáreas.
- **Ubicación:** 2 campos en Buenos Aires, 2 en Santa Fe y 2 en Córdoba (zona núcleo pampeana).
- **Sensores:** 1 estación IoT por campo con 5 sensores cada una (temperatura, humedad ambiental, humedad del suelo, pluviómetro y anemómetro): 30 sensores en total que envían una lectura cada 15 minutos, unos 576 registros por día.
- **Usuarios de los datos:** ingenieros agrónomos, jefes de campo, el área de planificación y la dirección.

2.2 Problema a resolver

Los datos actuales presentan problemas de calidad: valores faltantes, duplicados, mediciones fuera de rango, inconsistencias y períodos sin información. Esto afecta la confiabilidad de los dashboards que utiliza la organización para tomar decisiones.

Magnitud del problema (estimaciones iniciales, a validar en el relevamiento):

- Alrededor del 8% de los registros diarios presenta algún error (faltantes, duplicados, valores fuera de rango o unidades incorrectas).
- Un analista dedica unas 10 horas por semana a revisar y corregir datos a mano en planillas, sin registro de qué se cambió.
- Las fallas de un sensor se detectan recién cuando alguien nota el dashboard vacío, a veces días después.

Ejemplo concreto: durante un fin de semana el pluviómetro de uno de los campos dejó de transmitir y el dashboard mostró 0 mm de lluvia. Con ese dato se programó un riego que no hacía falta, porque en realidad habían caído 25 mm. El error solo se descubrió al revisar los datos de otra fuente.

2.3 Sistema propuesto y nivel de criticidad

Se propone un pipeline DataOps que centraliza los datos de distintas fuentes, ejecuta reglas de calidad automáticas, separa los datos válidos de los rechazados y publica únicamente datos validados para los dashboards.

Nivel de criticidad: Alta. Los dashboards alimentados por este pipeline se usan para decidir el riego, las fechas de siembra, las ventanas de pulverización (que dependen del viento y la lluvia) y la planificación de cada campaña.

Si el pipeline dejara de funcionar, los dashboards quedarían desactualizados y las decisiones se tomarían con datos viejos o sin datos; si publicara datos erróneos, el impacto sería peor, porque llevaría a decisiones equivocadas sobre lotes completos. No afecta la seguridad de las personas ni controla maquinaria (está fuera de alcance), por eso no se requiere alta disponibilidad para la POC.

3. Objetivo

3.1 Objetivo general

Diseñar e implementar un pipeline DataOps que garantice la calidad y confiabilidad de los datos agroclimáticos provenientes de dispositivos IoT, automatizando su validación, transformación, monitoreo y despliegue mediante prácticas de CI/CD.

3.2 Objetivos específicos

- Centralizar los datos provenientes de distintas fuentes.
- Detectar automáticamente problemas de calidad.
- Clasificar y registrar los errores encontrados.
- Separar datos válidos de datos rechazados.
- Automatizar las validaciones mediante tests.
- Implementar un esquema de procesamiento Medallion (Bronze, Silver, Gold).
- Automatizar el pipeline mediante CI/CD.
- Generar métricas de calidad.
- Permitir que los dashboards consuman únicamente datos validados.

Metas medibles (SMART) — todas se verifican en la POC al cierre de la semana 6 (10/11/2026):

- **Centralización:** el 100% de los datos de Open-Meteo y de las 6 estaciones IoT simuladas se carga en Bronze con origen y fecha de ingesta, a partir de la semana 4.
- **Detección:** las 12 reglas de calidad se ejecutan automáticamente en cada corrida y detectan al menos el 95% de los errores inyectados por el simulador.
- **Registro y separación:** el 100% de los registros rechazados queda en quarantine con su motivo, y la capa Gold tiene 0 registros inválidos (verificado por un test automatizado).
- **Tests:** al menos un test automatizado por regla (12 o más) y cobertura mínima del 80% en el módulo de validación.
- **CI/CD:** el 100% de los cambios a la rama principal pasa por GitHub Actions (lint + tests) antes de integrarse.
- **Métricas y consumo:** Data Quality Score calculado por campo y por día, y dashboard conectado solo a la capa Gold, en la semana 6.
- **Rendimiento:** una corrida completa del pipeline (un día de datos) termina en menos de 5 minutos.

4. Resumen de requerimientos

A continuación se presenta un resumen de los requerimientos del sistema.

4.1 Requerimientos funcionales

ID	Nombre	Descripción	Prioridad
RF01	Ingesta	Obtener periódicamente datos meteorológicos desde la Open-Meteo Historical Weather API y datos simulados de sensores IoT.	Alta
RF02	Almacenamiento raw	Conservar los datos originales tal como fueron recibidos.	Alta
RF03	Validación	Ejecutar automáticamente las reglas de calidad definidas.	Alta
RF04	Clasificación	Clasificar los registros como válidos, inválidos o con warning.	Alta
RF05	Rechazo	Separar del dataset válido los registros que incumplan reglas críticas.	Alta
RF06	Transformación	Transformar los datos validados al formato requerido para su consumo.	Media
RF07	Métricas	Calcular indicadores de calidad. Data Quality Score = registros válidos / registros totales × 100.	Alta

ID	Nombre	Descripción	Prioridad
RF08	Registro de errores	Registrar los errores detectados (fecha, campo, variable, regla incumplida, severidad).	Media
RF09	CI/CD	Ejecutar automáticamente los tests ante cada modificación del código.	Alta
RF10	Dashboard	Disponibilizar los datos validados para su visualización.	Media

4.2 Requerimientos no funcionales

ID	Categoría	Descripción	Prioridad
RNF01	Automatización	Las tareas de ingesta, validación y transformación se ejecutan automáticamente.	Alta
RNF02	Trazabilidad	Debe poder identificarse el origen y la fecha de cada dato.	Media
RNF03	Reproducibilidad	El pipeline debe poder ejecutarse nuevamente sobre los datos originales.	Media
RNF04	Calidad	Los datos que no cumplan reglas críticas no deben llegar a la capa Gold.	Alta
RNF05	Versionado	El código y las reglas de calidad están versionados mediante Git.	Alta
RNF06	Mantenibilidad	Las reglas pueden agregarse o modificarse sin rediseñar el pipeline.	Media
RNF07	Seguridad	Las credenciales (base de datos, APIs) se manejan solo con variables de entorno y GitHub Secrets; el archivo .env no se versiona y el acceso a la base es con un usuario de permisos mínimos.	Alta
RNF08	Rendimiento	Una corrida completa del pipeline sobre un día de datos debe terminar en menos de 5 minutos.	Baja

Criterio de prioridad: Alta = imprescindible para que la POC cumpla su objetivo; Media = necesario para una entrega completa, pero se puede simplificar; Baja = deseable si el tiempo lo permite.

5. Definición del proyecto

5.1 Alcance

El proyecto es una prueba de concepto (POC) de seis semanas. Incluye:

- Ingesta: obtención de datos desde Open-Meteo Historical Weather API, simulación de datos de sensores IoT e ingesta periódica.
- Procesamiento: almacenamiento de datos raw, limpieza y transformación, validación de calidad, detección de anomalías y clasificación de registros válidos e inválidos.
- Data Quality: definición y ejecución automática de reglas, registro de errores y métricas de calidad.
- DataOps / CI-CD: repositorio Git, tests automatizados, pipeline CI/CD, automatización de la ejecución, control de versiones y deploy del pipeline.
- Consumo: dataset final para dashboards e indicadores de calidad de los datos.

5.2 Fuera de alcance

- Desarrollo de una aplicación móvil.
- Control físico de sensores IoT.
- Automatización de maquinaria agrícola.
- Predicción de cosechas mediante Machine Learning.
- Sistema productivo de alta disponibilidad.
- Integración con todos los sistemas reales de la organización.
- Gestión de procesos y operaciones de calidad de datos (incidentes, cambios y problemas).

5.3 Mapa de stakeholders

Identificación de los actores clave del proyecto, clasificados según su nivel de interés e influencia, con una breve estrategia de comunicación para cada grupo.

Stakeholder	Tipo	Interés	Poder	Estrategia	Comunicación
Director/a de Producción (sponsor)	Interno (cliente)	Alto	Alto	Gestionar de cerca	Reunión quincenal de avance e informe de hitos por mail.
Equipo UnaCocuCha	Equipo del proyecto	Alto	Alto	Gestionar de cerca	Reunión semanal, tablero Kanban y chat del equipo (diario).
Gerente de IT del cliente	Interno (cliente)	Medio	Alto	Mantener satisfecho	Revisión de arquitectura, accesos y seguridad en cada hito técnico.
Proveedor de sensores IoT	Externo (proveedor)	Bajo	Medio	Mantener satisfecho	Mail ante fallas de hardware o cambios en el formato de los datos.
Ingenieros agrónomos y jefes de campo	Interno (usuarios)	Alto	Medio	Mantener informado	Taller para validar rangos de las reglas y demo en cada hito.
Área de Planificación	Interno (área afectada)	Medio	Medio	Mantener informado	Informe mensual y demo final del dashboard.
Analista de datos del cliente	Interno (usuario técnico)	Alto	Bajo	Mantener informado	Canal de Teams y sesión semanal; recibe la transferencia de conocimiento.
Open-Meteo	Externo (proveedor de datos)	Bajo	Bajo	Monitorear	Seguimiento de la documentación, cambios y límites de uso de la API.



Figura 1. Matriz poder–interés de los stakeholders.

Lectura: el sponsor y el equipo consultor concentran el poder de decisión y el interés, por eso se gestionan de cerca. Los agrónomos y el analista del cliente tienen alto interés pero poca capacidad de decidir: se los mantiene informados y participan validando las reglas de calidad.

5.4 Entregables esperados

- Documento de proyecto (este documento).
- Repositorio Git con el código del pipeline, las reglas de calidad y los tests.
- Pipeline CI/CD funcionando (GitHub Actions).
- Base de datos con capas Bronze, Silver y Gold.
- Dataset final y dashboard con indicadores de calidad.
- Tablero Kanban con evidencia de avance.
- Presentación grupal (15 a 20 minutos) con demo o video.

6. Arquitectura de la solución

6.1 Esquema Medallion

Se utiliza un esquema de tres capas: Fuentes (Open-Meteo API y sensores IoT) → Bronze → Silver → Gold → Dashboards.

Capa	Contenido	Objetivo
Bronze (raw)	Datos tal como fueron recibidos: timestamp, field_id, latitude, longitude, temperature, humidity, soil_moisture, precipitation, wind_speed, source. No se corrigen.	Conservar la fuente original y permitir reprocesar.

Capa	Contenido	Objetivo
Silver (clean & validated)	Ejecución de las reglas de calidad. Cada registro se clasifica como VALID, INVALID o WARNING. Los inválidos van a una estructura de quarantine / rejected data.	Aplicar Data Quality sin contaminar los datos buenos.
Gold (business)	Datos agregados por campo y fecha: temperatura promedio, precipitación total, humedad de suelo y ambiental promedio, cantidad de mediciones, porcentaje de calidad.	Capa que consumen los dashboards.

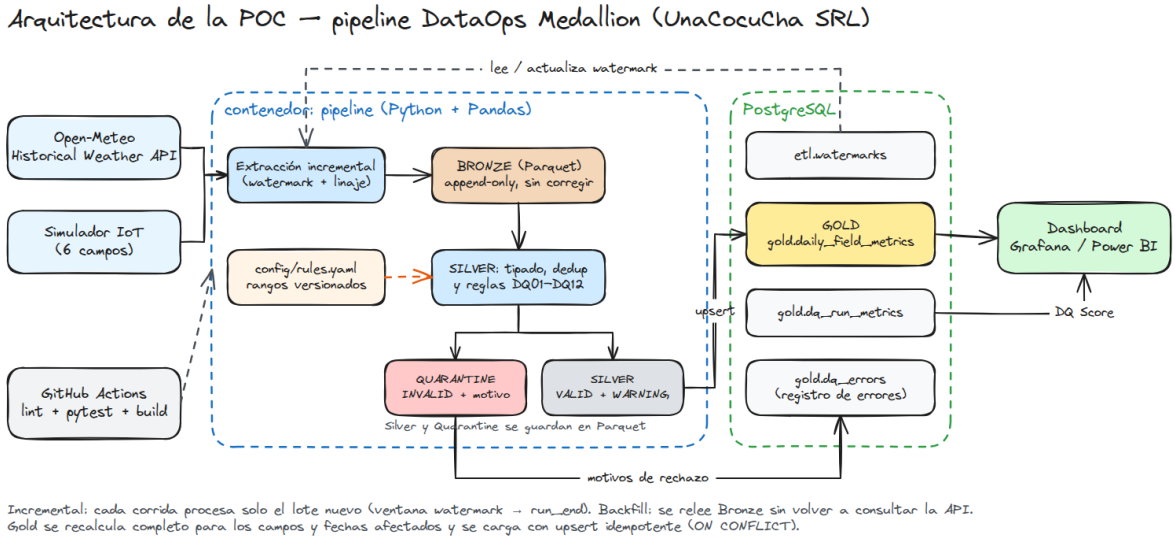


Figura 2. Arquitectura de la POC: fuentes, capas Medallion, PostgreSQL, dashboard y CI/CD.

Cómo fluye un dato:

- **Extracción incremental:** cada corrida lee el último watermark y trae solo los datos nuevos de Open-Meteo y del simulador IoT, agregando linaje (_batch_id, _ingested_at, _source_system).
- **Bronze:** se guarda el lote tal cual llegó en Parquet, sin modificarlo nunca (append-only). Permite reprocesar sin volver a llamar a la API.
- **Silver:** se tipan los datos, se eliminan duplicados y se ejecutan las 12 reglas. Los registros VALID y WARNING van a Silver; los INVALID a Quarantine con el motivo de rechazo.
- **Gold:** se recalculan los indicadores diarios de los campos y fechas afectados y se cargan en PostgreSQL con upsert idempotente. Los rechazos se registran en gold.dq_errors y el resumen de la corrida en gold.dq_run_metrics.
- **Consumo y CI/CD:** el dashboard lee solo tablas Gold. Cada cambio de código pasa por GitHub Actions (lint, tests y build de la imagen Docker).

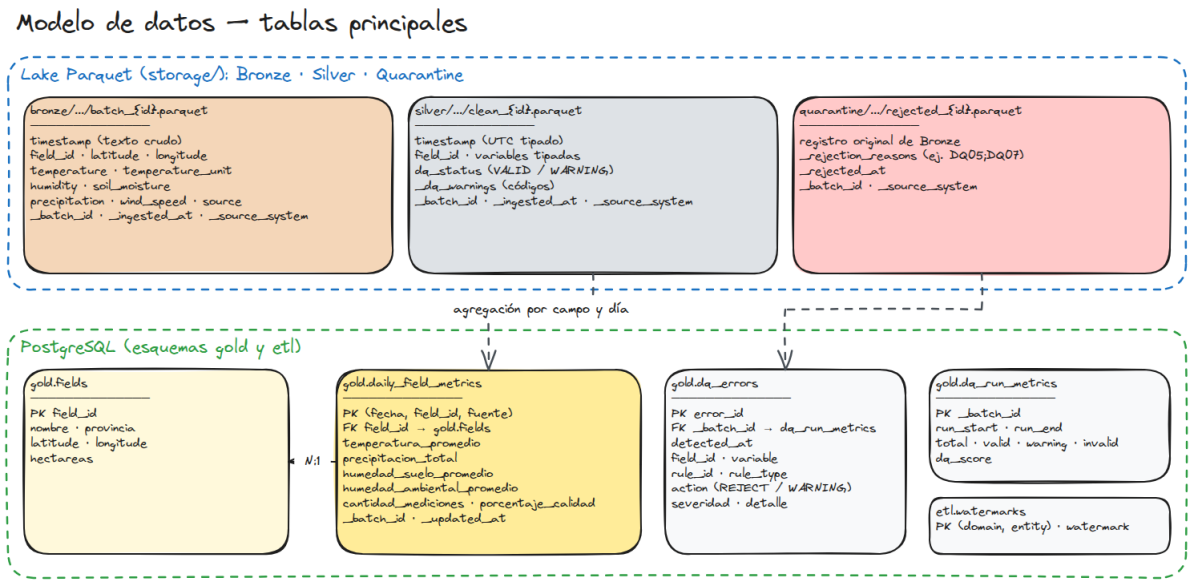


Figura 3. Modelo de datos: archivos Parquet del lake y tablas de PostgreSQL.

6.2 Reglas de Data Quality

ID	Regla	Tipo	Criticidad	Rango inicial / criterio	Acción
DQ01	timestamp obligatorio	Completitud	Crítica	No nulo ni vacío	Rechazar
DQ02	field_id obligatorio	Completitud	Crítica	No nulo ni vacío	Rechazar
DQ03	Timestamp con formato válido	Consistencia	Crítica	ISO 8601 en UTC; no futuro (tolerancia 10 min); posterior al año 2000	Rechazar
DQ04	La medición pertenece a un campo existente	Integridad	Crítica	field_id presente en gold.fields (C01 a C06)	Rechazar
DQ05	Sin registros duplicados	Unicidad	Crítica	Clave (source, field_id, timestamp) única; se conserva el primero	Rechazar
DQ06	Unidades esperadas	Consistencia	Crítica	Temperatura °C · humedad % · suelo m³/m³ · lluvia mm · viento km/h	Rechazar
DQ07	Humedad ambiental entre 0% y 100%	Validez	Alta	0 ≤ humedad ≤ 100	Rechazar
DQ08	Precipitación ≥ 0	Validez	Alta	precipitación ≥ 0 mm	Rechazar
DQ09	Temperatura dentro de rango	Validez	Alta	Rechazo fuera de −40 a 60 °C; warning fuera de −15 a 45 °C	Rechazar / Warning
DQ10	Humedad del suelo dentro de rango	Validez	Alta	Rechazo fuera de 0 a 1 m³/m³; warning fuera de 0,02 a 0,60 m³/m³	Warning / Rechazar
DQ11	Sin períodos inesperados sin datos	Completitud	Media	Hueco ≥ 30 min (2 lecturas seguidas de 15 min) por campo y variable	Warning
DQ12	Sin cambios anormales entre mediciones consecutivas	Anomalía	Media	Temperatura > 8 °C/h · humedad > 40 puntos/h · suelo > 0,15 m³/m³ por hora	Warning

Orden por criticidad: Crítica = el registro no se puede usar ni identificar (bloquea siempre); Alta = el valor es físicamente imposible o inconsistente; Media = el dato se publica pero queda marcado para revisión.

Rangos iniciales: son valores de arranque para la zona pampeana, pensados para que la POC funcione. Se validan con los ingenieros agrónomos del cliente en el taller de reglas de la semana 2 y quedan en el archivo config/rules.yaml, versionado en Git: cambiar un umbral no requiere modificar el código (RNF06).

7. Estructura del proyecto

7.1 Desglose del trabajo (EDT)

Identificación de tareas principales, sub-tareas y responsables.

Código	Tarea / sub-tarea	Responsable	Semana
1	Gestión del proyecto	PM	S1–S6
1.1	Plan de proyecto, EDT y cronograma	PM	S1–S2
1.2	Tablero Kanban en ClickUp y seguimiento semanal	PM	S1–S6
1.3	Riesgos, recursos y costos	PM	S4–S5
2	Relevamiento y definición de reglas	QA / Data Quality	S1–S2
2.1	Relevamiento del problema y stakeholders	PM + QA	S1
2.2	Taller de reglas y rangos con los agrónomos	QA / Data Quality	S2
3	Diseño de arquitectura	Data Engineer	S2–S3
3.1	Arquitectura Medallion y modelo de datos	Data Engineer	S2
3.2	Estructura del repositorio y convenciones Git	DevOps / CI-CD	S2–S3
4	Ingesta (API y simulador)	Data Engineer	S3–S4
4.1	Conector Open-Meteo con carga incremental (watermark)	Data Engineer	S3
4.2	Simulador de sensores IoT con errores inyectados	Data Engineer	S3
4.3	Escritura en Bronze (Parquet + linaje)	Data Engineer	S4
5	Capas Bronze / Silver / Gold	Data Engineer	S4–S5
5.1	Silver: tipado, deduplicación y quarantine	Data Engineer	S4
5.2	Gold: indicadores diarios y upsert en PostgreSQL	Data Engineer	S5
6	Tests y reglas de calidad	QA / Data Quality	S3–S5
6.1	Implementación de las reglas DQ01–DQ12	QA / Data Quality	S3–S4
6.2	Tests Pytest por regla y de punta a punta	QA / Data Quality	S4–S5
6.3	Métricas: Data Quality Score y registro de errores	QA / Data Quality	S5
7	CI/CD	DevOps / CI-CD	S3–S6
7.1	Docker y docker-compose (PostgreSQL + pipeline)	DevOps / CI-CD	S3
7.2	GitHub Actions: lint, tests y build de la imagen	DevOps / CI-CD	S4
7.3	Ejecución programada y deploy del pipeline	DevOps / CI-CD	S5–S6
8	Dashboard	QA / Data Quality	S5–S6
8.1	Dashboard sobre Gold con indicadores de calidad	QA / Data Quality	S5–S6
9	Documentación y presentación	PM + todos	S5–S6
9.1	Documento final de proyecto	PM	S5–S6

Código	Tarea / sub-tarea	Responsable	Semana
9.2	Video demo y slides	Todos	S6
9.3	Ensayo de la presentación oral	Todos	S6

Semanas: S1 = 29/9 · S2 = 6/10 · S3 = 13/10 · S4 = 20/10 · S5 = 27/10 · S6 = 3/11 al 10/11. Los paquetes (en gris) agrupan sus sub-tareas; cada sub-tarea es una tarjeta del tablero Kanban.

7.2 Cronograma

Duración total: 6 semanas (del 29/9/2026 al 10/11/2026).

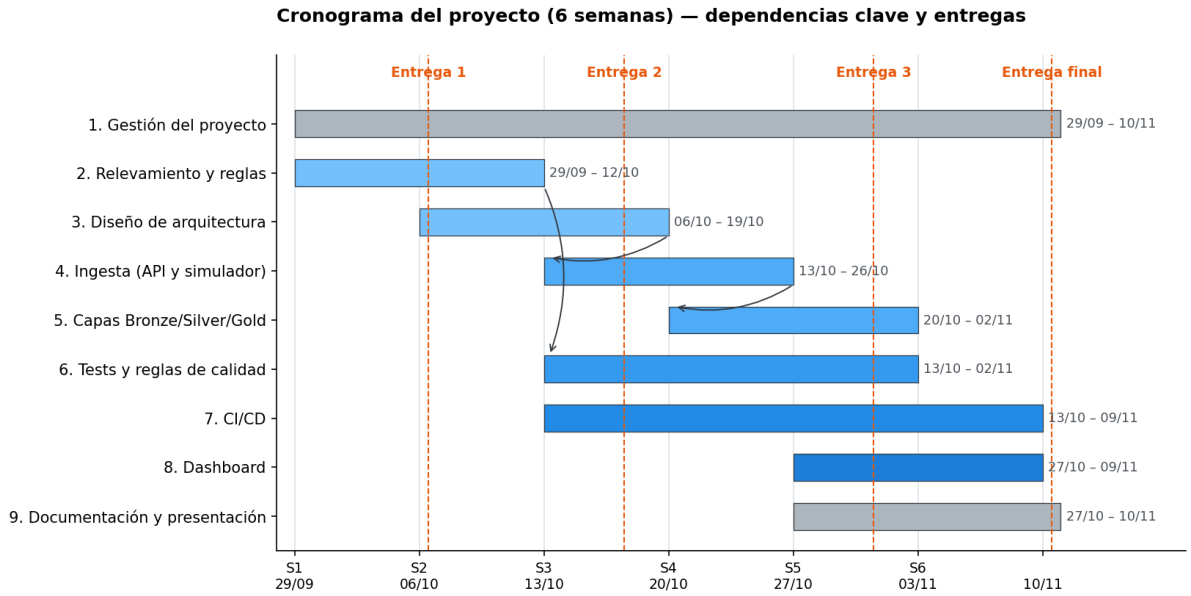


Figura 4. Diagrama de Gantt con las dependencias clave y las entregas del TP.

Dependencias clave: las reglas (6) se implementan después del taller con los agrónomos (2); la ingesta (4) necesita la arquitectura definida (3); las capas Silver y Gold (5) se construyen sobre la ingesta (4); el dashboard (8) se conecta cuando Gold tiene datos. CI/CD (7) arranca temprano para que todo el código nuevo ya pase por los tests.

7.3 Hitos y fechas de entrega

Hito	Fecha
Entrega parcial 1: tema, posicionamiento del equipo, objetivos y mapa de stakeholders (borrador)	Semana del 29/9 al 6/10
Entrega parcial 2: EDT, cronograma preliminar y tablero Kanban en funcionamiento	Semana del 13/10 al 17/10
Hito técnico: repositorio, Docker y pipeline de CI (lint + tests) funcionando	19/10
Hito técnico: ingesta de Open-Meteo y del simulador cargando Bronze	23/10
Hito técnico: capa Silver con las 12 reglas y quarantine	26/10
Entrega parcial 3: riesgos y mitigación, recursos y costos, herramientas seleccionadas, avance del documento	Semana del 27/10 al 31/10
Hito técnico: Gold en PostgreSQL y dashboard conectado	2/11
Hito técnico: demo grabada y documento en revisión final	7/11
Entrega final: documento de proyecto + presentación oral	10 de noviembre de 2026

Referencia: en azul, los hitos técnicos internos del equipo; el resto son las entregas que pide la cátedra.

7.4 Tablero Kanban de seguimiento

Las tareas de la EDT se organizan en un tablero con las columnas: Por hacer / En curso / A validar / Completado, actualizado semanalmente durante las 6 semanas.

Herramienta: ClickUp, con una vista de tablero por estado. Cada sub-tarea de la EDT es una tarjeta con responsable, semana y etiqueta del paquete de trabajo (1 a 9).

- **Columnas:** Por hacer → En curso → A validar → Completado.
- **Regla de trabajo:** una tarjeta pasa a Completado solo si otro integrante la revisó (en el código, con un pull request aprobado).
- **Evidencia:** captura del tablero cada viernes, guardada en la carpeta docs/kanban del repositorio y agregada como anexo.

Link al tablero: [agregar link de ClickUp]

8. Recursos y costos

8.1 Recursos

Tipo	Recurso	Detalle
Humanos	4 integrantes: Project Manager, Data Engineer, QA / Data Quality y DevOps / CI-CD	430 horas en 6 semanas: PM 80 h · Data Engineer 140 h · QA 110 h · DevOps 100 h
Tecnológicos	Python 3.11, Pandas, PyArrow (Parquet), PostgreSQL 16, Pytest, Docker, GitHub + GitHub Actions, Grafana (o Power BI), Open-Meteo API, ClickUp	Todos gratuitos u open source. La POC corre en forma local con Docker: no hay costo de nube.
Materiales	4 notebooks del equipo, conexión a internet, cuentas de GitHub y ClickUp	Equipos propios del equipo; se imputa un costo de uso prorrateado.

Supuesto de horas: dedicación de 15 a 25 horas semanales por integrante durante 6 semanas, según la carga de cada rol en la EDT.

8.2 Estimación de costos

Concepto	Cantidad	Costo unitario	Total
Honorarios Project Manager	80 h	USD 30/h	USD 2.400
Honorarios Data Engineer	140 h	USD 28/h	USD 3.920
Honorarios QA / Data Quality	110 h	USD 22/h	USD 2.420
Honorarios DevOps / CI-CD	100 h	USD 28/h	USD 2.800
Licencias (GitHub, ClickUp, Grafana, PostgreSQL)	Planes gratuitos / open source	USD 0	USD 0
Infraestructura (POC local con Docker)	—	USD 0	USD 0
Uso de equipos y conectividad	4 personas × 2 meses	USD 25/mes	USD 200
Subtotal			USD 11.740
Contingencia (10%)			USD 1.174
TOTAL			USD 12.914

Supuestos: valores en dólares estadounidenses (USD), como suelen cotizar las consultoras de datos en Argentina; tarifas por hora según el rol; herramientas en sus planes

gratuitos; la POC corre local, por eso no hay costo de nube. Si el dashboard se hiciera en Power BI Pro, se suman USD 14 por usuario por mes.

Distribución de los honorarios por fase:

Fase	Horas	% del esfuerzo	Costo
Relevamiento y diseño (paquetes 2 y 3)	70 h	16%	USD 1.879
Construcción del pipeline (4, 5 y 6)	200 h	47%	USD 5.367
CI/CD y dashboard (7 y 8)	90 h	21%	USD 2.415
Gestión y documentación (1 y 9)	70 h	16%	USD 1.879
Total honorarios	430 h	100%	USD 11.540

Para producción (fuera de la POC): una VM chica en la nube para el pipeline y PostgreSQL costaría del orden de USD 40 a 60 por mes, a validar con el área de IT del cliente.

9. Gestión de riesgos

9.1 Metodología

Cada riesgo se evalúa con probabilidad e impacto en una escala de 1 (muy bajo) a 5 (muy alto). Exposición = Probabilidad × Impacto. Exposición 1-6: riesgo bajo (aceptar/monitorear); 8-12: medio (mitigar); 15-25: alto (mitigar o evitar de forma prioritaria).

9.2 Registro de riesgos

ID	Descripción	Prob.	Imp.	Expos.	Estrategia / mitigación	Dueño
R01	Rangos de calidad mal definidos: se rechazan datos buenos o pasan errores sin detectar	4	4	16	Mitigar: rangos iniciales documentados, taller con los agrónomos en S2 y reglas dudosas como Warning hasta calibrarlas.	QA
R02	Alcance excesivo (scope creep): pedidos de incidentes, ML o nuevas fuentes	4	3	12	Evitar: alcance y fuera de alcance escritos (5.1 y 5.2); todo cambio se acuerda con el sponsor.	PM
R03	Poca experiencia del equipo con CI/CD, Parquet o Great Expectations	3	4	12	Mitigar: CI mínimo desde S3, programación en pares, plantillas de código reutilizables y tests propios con Pytest.	DevOps
R04	Poca disponibilidad del equipo (parciales, trabajo)	4	3	12	Mitigar: un backup por tarea, seguimiento semanal en ClickUp y holgura en la semana 6.	PM
R05	Exposición de credenciales en el repositorio	2	5	10	Evitar: archivo .env fuera de Git, GitHub Secrets en el pipeline y escaneo de secretos activado.	DevOps
R06	Datos simulados poco realistas	3	3	9	Mitigar: calibrar el simulador con datos reales de Open-Meteo de la misma zona y validar con el cliente.	Data Engineer
R07	Errores detectados sin seguimiento formal (la	3	3	9	Aceptar: registro en gold.dq_errors y revisión	QA

ID	Descripción	Prob.	Imp.	Expos.	Estrategia / mitigación	Dueño
	gestión de incidentes queda fuera de alcance)				semanal; el proceso formal queda como recomendación.	
R08	Caídas, cambios o límites de uso de la API de Open-Meteo	2	4	8	Mitigar: Bronze conserva lo descargado, reintentos automáticos y backfill desde Bronze sin volver a llamar a la API.	Data Engineer
R09	Pipeline lento (más de 5 minutos por corrida)	2	2	4	Aceptar y monitorear: procesar solo el lote nuevo (watermark) y usar Parquet columnar.	Data Engineer

Colores: Prob. = probabilidad; Imp. = impacto; Expos. = exposición (Prob. × Imp.); Dueño = rol responsable. Rojo = riesgo alto (15 a 25); amarillo = medio (8 a 12); verde = bajo (1 a 6). El registro se revisa en la reunión semanal y se actualiza en ClickUp.

10. Herramientas y buenas prácticas

10.1 Stack tecnológico del pipeline

Componente	Tecnología propuesta
Fuente meteorológica	Open-Meteo Historical Weather API
Lenguaje	Python
Procesamiento	Pandas / Polars
Base de datos	PostgreSQL
Data Quality	Great Expectations o tests propios con Pytest
Versionado	Git + GitHub
CI/CD	GitHub Actions
Orquestación	Apache Airflow (opcional)
Dashboard	Power BI / Grafana
Contenedores	Docker
Documentación	Markdown / GitHub

Para mantener la POC simple, el stack inicial es: Python + PostgreSQL + Pytest + GitHub Actions + Docker + Power BI/Grafana. Airflow queda como componente opcional para demostrar orquestación.

[Completar] *Justificar cada elección (por qué esa herramienta y no otra) y decidir cuáles son obligatorias y cuáles opcionales.*

10.2 Herramientas de gestión y colaboración

[Completar] *Seleccionar y justificar las herramientas de gestión de proyectos y colaboración (Trello/Jira/Miro para Kanban, MS Project o similar para el Gantt, Teams/WhatsApp/Discord para comunicación, Drive/GitHub para documentación).*

10.3 Buenas prácticas de trabajo en equipo

[Completar] *Definir: repositorio compartido para la documentación, plan de comunicación interna, cadencia de reuniones de seguimiento (ej.: una semanal), registro de decisiones*

(actas o bitácora) y convenciones de Git (ramas por funcionalidad, revisión cruzada de código, commits descriptivos).

11. Prueba de concepto (POC)

[Completar] *Describir la POC que se mostrará en la demo: qué datos se ingestan, qué reglas se ejecutan, cómo se ve el resultado en cada capa, evidencia de un test que falla y luego pasa en el pipeline CI/CD, y el dashboard final. Incluir capturas o link a un video/repositorio.*

12. Conclusiones

12.1 Beneficios esperados

[Completar] *Exponer los beneficios de implementar el plan (datos confiables, menos corrección manual, decisiones más rápidas, trazabilidad, reproducibilidad).*

12.2 Limitaciones y recomendaciones

[Completar] *Reconocer limitaciones (datos simulados, alcance de POC, sin alta disponibilidad, sin gestión de incidentes formal) y recomendar próximos pasos (integrar sensores reales, incorporar orquestación, gestión de incidentes de calidad de datos, ampliar reglas).*